

PySpark Quick Revision

Scratch → Intermediate → Advanced → Production → Interview

1. What is PySpark?

PySpark is the Python API for **Apache Spark**, a distributed data-processing framework.

Spark is used to process large datasets across multiple machines.

Why PySpark?

Traditional Python/Pandas:

```
graph TD; A[Data] --> B[Single Machine]; B --> C[CPU / Memory limitation];
```

Data
↓
Single Machine
↓
CPU / Memory limitation

LinkedIn@AshishZope

PySpark:

```
graph TD; A[Large Data] --> B[Spark Cluster]; B --> C1[Node]; B --> C2[Node]; B --> C3[Node]; B --> C4[Node];
```

Large Data
↓
Spark Cluster
↓ ↓ ↓ ↓
Node Node Node Node

LinkedIn@AshishZope

Important characteristics

- Distributed processing
- In-memory computation
- Fault tolerance
- Lazy evaluation

- Parallel processing
- Supports SQL, DataFrames, Streaming, ML
- Works with AWS, Azure, GCP and Databricks

PySpark interview angle

Interviewers are usually testing whether you understand the difference between:

LinkedIn@AshishZope

Big data processing on a single machine vs cluster-based processing

Key idea:

- PySpark does not run computation in Python directly on the driver for all data
- It splits work into tasks and distributes them to executors
- Spark uses parallelism, fault tolerance, and optimization for large-scale processing

A good interview answer is:

PySpark is the Python interface to Apache Spark, which runs distributed data processing across a cluster. It allows developers to work with DataFrames/SQL in a high-level way while Spark handles parallel execution, task scheduling, and optimization underneath.

2. Apache Spark Architecture

Main components:

LinkedIn@AshishZope

```

Driver
|
|- Cluster Manager
|
|- Executors
    |
    |- Tasks
  
```

Driver

The driver:

- Creates SparkSession
- Builds execution plan
- Coordinates jobs
- Communicates with executors

Executor

Executors:

- Execute tasks
- Perform transformations/actions
- Store cached data
- Return results to driver

Cluster Manager

Allocates resources.

Examples:

- Spark Standalone
- YARN
- Kubernetes
- Databricks

Execution flow: Job → Stage → Task

When an action is triggered, Spark creates a logical execution plan and then a physical plan.

```
User code
  ↓
Transformation graph
  ↓
Job
  ↓
Stage(s)
  ↓
Task(s)
  ↓
Executor
```

Job

A job is created when an action like `show()`, `count()`, `save()` is called.

Stage

A stage is a set of tasks that can run without a shuffle.

Task

A task is the smallest unit of execution and runs on a single partition.

Interview point

A wide transformation usually creates a new stage because data must be shuffled across partitions.

Examples of wide transformations:

```
groupBy
join
distinct
orderBy
repartition
```

Narrow transformations can stay in the same stage:

```
filter  
select  
withColumn  
map
```

3. SparkSession

SparkSession is the entry point to Spark.

```
from pyspark.sql import SparkSession  
  
spark = SparkSession.builder \  
    .appName("EmployeeApp") \  
    .getOrCreate()
```

You normally **don't create a new SparkSession for every DataFrame operation.**

In Databricks, `spark` is generally already available.

```
df = spark.read.csv("employees.csv", header=True, inferSchema=True)
```

4. DataFrame

A DataFrame is a distributed collection of data organized into named columns.

Example:

```
df_emp = spark.read.csv(  
    "/FileStore/employees.csv",  
    header=True,  
    inferSchema=True  
)
```

Inspect:

```
df_emp.show()
df_emp.printSchema()
df_emp.columns
df_emp.dtypes
df_emp.count()
df_emp.describe().show()
```

LinkedIn@AshishZope

Why DataFrame is preferred over RDD

Modern Spark engineering mostly uses DataFrames because they provide:

- Structured data with schema
- Better optimization through Catalyst
- Easier SQL integration
- Cleaner code and easier debugging
- Performance benefits from Tungsten and code generation

RDDs are still useful in low-level/custom transformations, but for most real-world jobs, DataFrame is the standard choice.

5. Schema

Example:

```
from pyspark.sql.types import *

schema = StructType([
    StructField("employee_id", IntegerType(), False),
    StructField("first_name", StringType(), True),
    StructField("salary", DoubleType(), True)
])
```

LinkedIn@AshishZope

Read:

```
df = spark.read.schema(schema).csv(  
    "employees.csv",  
    header=True  
)
```

Why explicit schema?

Better than:

```
inferSchema=True
```

because explicit schemas:

- Improve performance
- Prevent incorrect type inference
- Make pipelines predictable
- Are preferred in production

Interview note

Schema design is part of production quality. A bad schema can silently change output and cause downstream errors. In pipelines, explicit schemas help with:

- Type safety
- Data quality checks
- Reproducibility
- Better optimization

6. Data Types

Common PySpark types:

```
StringType
IntegerType
LongType
DoubleType
FloatType
BooleanType
DateType
TimestampType
DecimalType
ArrayType
MapType
StructType
```

7. Selecting Columns

```
df_emp.select("first_name", "salary").show()
```

Using `col()`:

```
from pyspark.sql.functions import col

df_emp.select(
    col("first_name"),
    col("salary")
).show()
```

Alias:

```
df_emp.select(
    col("first_name").alias("f_name")
).show()
```

8. selectExpr()

Useful for SQL-style expressions.

```
df_emp.selectExpr(  
    "first_name as f_name",  
    "salary",  
    "salary * 1.10 as increased_salary"  
).show()
```

LinkedIn@AshishZope

Remember:

```
select()
```

LinkedIn@AshishZope

works mainly with column objects/column names.

```
selectExpr()
```

LinkedIn@AshishZope

accepts SQL expressions as strings.

9. withColumn()

Create or modify a column.

```
df_emp.withColumn(  
    "salary_10_percent",  
    col("salary") * 10 / 100  
).show()
```

LinkedIn@AshishZope

Increase salary:

LinkedIn@AshishZope

```
df_emp.withColumn(  
  "new_salary",  
  col("salary") * 1.10  
)
```

Rename:

LinkedIn@AshishZope

```
df_emp.withColumnRenamed(  
  "first_name",  
  "employee_first_name"  
)
```

10. filter() / where()

LinkedIn@AshishZope

```
df_emp.filter(  
  col("salary") > 700000  
)<div data-bbox="67 571 254 591" data-label="Text">

Multiple conditions:


```

LinkedIn@AshishZope

```
df_emp.filter(  
  (col("salary") > 700000) &  
  (col("salary") < 1500000)  
)<div data-bbox="67 757 107 774" data-label="Text">

OR:


```

LinkedIn@AshishZope

```
df_emp.filter(  
  (col("department") == "IT") |  
  (col("department") == "HR")  
)
```

Important

Don't write:

```
df_emp.filter(col("job_id") in (2,3))
```

LinkedIn@AshishZope

Correct:

```
df_emp.filter(  
    col("job_id").isin(2, 3)  
)
```

LinkedIn@AshishZope

Interview logic behind filter conditions

When writing filters in Spark, always remember:

- Use column expressions instead of Python comparisons that do not translate well to Spark internals
- Use `isin()` for membership checks
- Combine conditions with `&` and `|`, not Python `and/or`
- Use `when()` for conditional logic

Example:

```
filtered = df.filter(  
    (col("salary") > 500000) &  
    (col("department") == "IT")  
)
```

LinkedIn@AshishZope

11. Sorting

```
df_emp.orderBy("salary").show()
```

LinkedIn@AshishZope

Descending:

```
df_emp.orderBy(  
    col("salary").desc()  
) .show()
```

LinkedIn@AshishZope

Multiple columns:

```
df_emp.orderBy(  
    col("department"),  
    col("salary").desc()  
)
```

LinkedIn@AshishZope

12. distinct() and dropDuplicates()

```
df.distinct()
```

LinkedIn@AshishZope

Remove duplicate rows:

```
df.dropDuplicates()
```

LinkedIn@AshishZope

Based on columns:

```
df.dropDuplicates(["email"])
```

LinkedIn@AshishZope

13. Null Handling

Find nulls:

LinkedIn@AshishZope

```
df.filter(  
    col("salary").isNull()  
).show()
```

Not null:

LinkedIn@AshishZope

```
df.filter(  
    col("salary").isNotNull()  
)
```

Drop nulls:

LinkedIn@AshishZope

```
df.dropna()
```

Fill nulls:

LinkedIn@AshishZope

```
df.fillna(0)
```

Specific column:

LinkedIn@AshishZope

```
df.fillna(  
    {"salary": 0}  
)
```

14. when() / otherwise()

SQL CASE WHEN equivalent.

```
from pyspark.sql.functions import when

df.withColumn(
    "salary_category",
    when(col("salary") > 1000000, "High")
    .when(col("salary") > 500000, "Medium")
    .otherwise("Low")
)
```

15. Common String Functions

```
from pyspark.sql.functions import *

upper("name")
lower("name")
trim("name")
length("name")
substring("name", 1, 3)
concat("first_name", "last_name")
concat_ws(" ", "first_name", "last_name")
regexp_replace("name", "old", "new")
split("name", " ")
```

Example:

```
df.withColumn(
    "full_name",
    concat_ws(" ", "first_name", "last_name")
)
```

16. Date Functions

```
current_date()
current_timestamp()
year()
month()
day()
datediff()
date_add()
date_sub()
to_date()
date_format()
```

Example:

```
df.withColumn(
    "joining_year",
    year("joining_date")
)
```

Date difference:

```
df.withColumn(
    "experience_days",
    datediff(current_date(), "joining_date")
)
```

17. Aggregations

```
df.agg(
    avg("salary").alias("avg_salary"),
    max("salary").alias("max_salary"),
    min("salary").alias("min_salary"),
    sum("salary").alias("total_salary"),
    count("*").alias("employee_count")
).show()
```

18. groupBy()

LinkedIn@AshishZope

```
df.groupBy("department") \
    .count() \
    .show()
```

Multiple aggregations:

LinkedIn@AshishZope

```
df.groupBy("department").agg(
    count("*").alias("employee_count"),
    avg("salary").alias("avg_salary"),
    max("salary").alias("max_salary")
).show()
```

19. Window Functions

Very important for interviews.

LinkedIn@AshishZope

```
from pyspark.sql.window import Window

window_spec = Window.partitionBy(
    "department"
).orderBy(
    col("salary").desc()
)
```

row_number()

LinkedIn@AshishZope

```
df.withColumn(
    "rn",
    row_number().over(window_spec)
)
```

rank()

LinkedIn@AshishZope

```
rank().over(window_spec)
```

dense_rank()

LinkedIn@AshishZope

```
dense_rank().over(window_spec)
```

Difference:

LinkedIn@AshishZope

```
salary  
100  
100  
90
```

rank:

LinkedIn@AshishZope

```
1  
1  
3
```

dense_rank:

LinkedIn@AshishZope

```
1  
1  
2
```

row_number:

LinkedIn@AshishZope

```
1  
2  
3
```

20. Top N Employees Per Department

Classic interview question.

LinkedIn@AshishZope

```
window_spec = Window.partitionBy(
    "department"
).orderBy(
    col("salary").desc()
)

result = df.withColumn(
    "rn",
    row_number().over(window_spec)
).filter(
    col("rn") <= 3
)
```

21. Running Total

LinkedIn@AshishZope

```
window_spec = Window \
    .partitionBy("department") \
    .orderBy("joining_date") \
    .rowsBetween(
        Window.unboundedPreceding,
        Window.currentRow
    )

df.withColumn(
    "running_salary",
    sum("salary").over(window_spec)
)
```

22. Lag and Lead

LinkedIn@AshishZope

```
lag("salary", 1).over(window_spec)
```

```
lead("salary", 1).over(window_spec)
```

Useful for:

- Previous transaction
- Next transaction
- Salary comparison
- Event analysis

23. Joins

Main types:

```
INNER  
LEFT  
RIGHT  
FULL  
LEFT SEMI  
LEFT ANTI  
CROSS
```

Example:

```
df_emp.join(  
    df_dept,  
    df_emp.department_id == df_dept.department_id,  
    "inner"  
)
```

24. Left Join

LinkedIn@AshishZope

```
df_emp.join(  
    df_dept,  
    "department_id",  
    "left"  
)
```

25. Left Semi Join

Returns rows from the left DataFrame where a match exists.

LinkedIn@AshishZope

```
df_emp.join(  
    df_dept,  
    "department_id",  
    "left_semi"  
)
```

Think:

Give me employees whose department exists.

LinkedIn@AshishZope

26. Left Anti Join

Returns rows from left where no match exists.

LinkedIn@AshishZope

```
df_emp.join(  
    df_dept,  
    "department_id",  
    "left_anti"  
)
```

Think:

Give me employees whose department does NOT exist.

Very useful for data-quality checks.

27. Broadcast Join

If one table is small:

```
from pyspark.sql.functions import broadcast

df_large.join(
    broadcast(df_small),
    "department_id"
)
```

Why?

Normally Spark may shuffle both datasets.

Broadcast:

```
Small table
  ↓
All Executors

Large table
  ↓
Processed locally
```

This can reduce expensive shuffle.

28. Narrow vs Wide Transformations

Narrow transformation

No data movement between partitions.

Examples:

```
map  
filter  
select  
withColumn
```

LinkedIn@AshishZope

Wide transformation

Requires shuffle.

Examples:

```
groupBy  
join  
distinct  
orderBy  
repartition
```

LinkedIn@AshishZope

Interview keyword:

Shuffle = expensive data movement between partitions.

29. Transformations vs Actions

Transformations

Lazy.

Examples:

```
select()  
filter()  
withColumn()  
groupBy()  
join()
```

Actions

Trigger execution.

Examples:

```
show()  
count()  
collect()  
first()  
take()  
write()
```

30. Lazy Evaluation

Spark doesn't immediately execute transformations.

Example:

```
df2 = df.filter(col("salary") > 500000)  
df3 = df2.select("name", "salary")
```

Nothing substantial executes yet.

When:

```
df3.show()
```

Spark builds and executes the plan.

Benefits:

- Optimization
- Predicate pushdown
- Column pruning
- Reduced unnecessary computation

31. Catalyst Optimizer

Spark SQL uses **Catalyst Optimizer**.

It optimizes query plans.

Example:

```
graph TD; A[Logical Plan] --> B[Analyzed Logical Plan]; B --> C[Optimized Logical Plan]; C --> D[Physical Plan];
```

Logical Plan
↓
Analyzed Logical Plan
↓
Optimized Logical Plan
↓
Physical Plan

LinkedIn@AshishZope

Common optimizations:

- Predicate pushdown
- Projection/column pruning
- Constant folding
- Join optimization

32. Tungsten

Tungsten improves Spark execution efficiency.

Focus areas:

- Memory management
- CPU efficiency
- Binary processing
- Whole-stage code generation

Interview distinction:

LinkedIn@AshishZope

Catalyst → query optimization

Tungsten → execution efficiency

33. Explain Plan

Extremely important for debugging.

LinkedIn@AshishZope

```
df.explain()
```

Detailed:

LinkedIn@AshishZope

```
df.explain(True)
```

Look for:

LinkedIn@AshishZope

```
Exchange
BroadcastHashJoin
SortMergeJoin
Filter
Project
```

Exchange

Usually indicates shuffle.

Physical vs logical plan

When using `explain()`, remember:

```
Logical Plan = what you want  
Physical Plan = how Spark will execute it
```

LinkedIn@AshishZope

This matters in interviews because they want to see you understand that Spark does not simply execute the Python code line by line. It builds plans and chooses the most efficient execution strategy.

34. Partitions

Spark divides data into partitions.

```
Dataset  
↓  
Partition 1  
Partition 2  
Partition 3  
Partition 4
```

LinkedIn@AshishZope

Each partition can be processed independently.

35. repartition()

Changes partition count and usually causes shuffle.

```
df.repartition(10)
```

LinkedIn@AshishZope

By column:

```
df.repartition(  
    10,  
    "department"  
)
```

Use when you intentionally want redistribution.

36. coalesce()

Reduces partitions with less/no full shuffle.

```
df.coalesce(2)
```

Common use:

```
Many partitions  
  ↓  
coalesce()  
  ↓  
Fewer partitions
```

Interview

repartition → can increase/decrease, shuffle

coalesce → generally used to decrease, avoids full shuffle

When to use which?

Use `repartition()` when:

- You want more parallelism
- You need even distribution before a heavy transformation

- You need to increase partitions for write performance

Use `coalesce()` when:

- You already have too many small partitions
- You want to reduce partition count without a full shuffle

37. collect() Warning

```
df.collect()
```

LinkedIn@AshishZope

Brings all data to the driver.

Danger:

```
Huge Data
  ↓
Driver Memory
  ↓
OOM
```

LinkedIn@AshishZope

Avoid `collect()` on large datasets.

Instead:

```
df.show(20)
df.take(10)
```

LinkedIn@AshishZope

38. Cache and Persist

If a DataFrame is reused:

LinkedIn@AshishZope

```
df.cache()
```

or:

LinkedIn@AshishZope

```
df.persist()
```

Example:

LinkedIn@AshishZope

```
df.cache()  
  
df.count()  
df.groupBy("department").count().show()
```

The first action materializes the cache.

Check:

LinkedIn@AshishZope

```
df.is_cached
```

Remove:

LinkedIn@AshishZope

```
df.unpersist()
```

39. cache() vs persist()

LinkedIn@AshishZope

```
cache()
```

uses default storage level.

```
persist()
```

allows specifying storage level.

Conceptually:

```
cache = convenient default  
persist = configurable storage
```

40. UDF

User Defined Function.

```
from pyspark.sql.functions import udf  
from pyspark.sql.types import StringType  
  
@udf(StringType())  
def salary_category(salary):  
    if salary > 1000000:  
        return "High"  
    return "Low"
```

Use:

```
df.withColumn(  
    "category",  
    salary_category("salary")  
)
```

Why avoid UDFs when possible?

Built-in Spark functions are generally:

- Faster

- Optimizable
- Better integrated with Catalyst

Prefer:

```
when()  
regexp_replace()  
concat()
```

LinkedIn@AshishZope

over Python UDF when possible.

Interview angle on UDFs

A strong answer is:

UDFs are useful when Spark does not have a built-in function for a business rule, but they are slower because they move row-level logic into Python. For performance-sensitive workloads, use native Spark functions whenever possible.

41. Pandas UDF

Also called vectorized UDF.

Uses Apache Arrow for efficient transfer between Spark and Python.

Useful when normal Spark functions cannot solve the problem efficiently.

42. RDD

RDD = Resilient Distributed Dataset.

Older Spark abstraction.

```
rdd = spark.sparkContext.parallelize(
    [1,2,3,4,5]
)
```

Modern Data Engineering generally prefers:

```
DataFrame
  ↓
Spark SQL
```

over manually working with RDDs.

Also remember your Databricks experience:

Some Serverless compute environments do not support arbitrary RDD APIs.

43. DataFrame vs RDD

DataFrame	RDD	Structured	Unstructured	Schema	No schema requirement
Catalyst optimization	Less SQL optimization	Easier	More low-level	Preferred for DE	Used for specialized cases

44. Reading Data

CSV

```
df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv(path)
```

JSON

LinkedIn@AshishZope

```
df = spark.read.json(path)
```

Parquet

LinkedIn@AshishZope

```
df = spark.read.parquet(path)
```

Delta

LinkedIn@AshishZope

```
df = spark.read.format("delta").load(path)
```

45. Writing Data

LinkedIn@AshishZope

```
df.write \  
  .mode("overwrite") \  
  .parquet(path)
```

Modes:

LinkedIn@AshishZope

```
append  
overwrite  
ignore  
error / errorifexists
```

46. Parquet

Columnar file format.

Advantages:

- Column pruning
- Compression
- Predicate pushdown
- Efficient analytics
- Schema information

For analytical workloads:

CSV < Parquet

LinkedIn@AshishZope

in many common scenarios.

47. Delta Lake

Delta Lake adds reliability and transaction capabilities on top of data lakes.

Important features:

- ACID transactions
- Schema enforcement
- Schema evolution
- Time travel
- MERGE
- UPDATE
- DELETE
- Version history

48. Delta Table

Example:

LinkedIn@AshishZope

```
df.write \  
  .format("delta") \  
  .mode("overwrite") \  
  .save("/delta/employees")
```

Read:

LinkedIn@AshishZope

```
df = spark.read \  
  .format("delta") \  
  .load("/delta/employees")
```

49. Delta Time Travel

Read an older version:

LinkedIn@AshishZope

```
df = spark.read \  
  .format("delta") \  
  .option("versionAsOf", 5) \  
  .load(path)
```

Using timestamp:

LinkedIn@AshishZope

```
.option(  
  "timestampAsOf",  
  "2026-10-01 10:00:00"  
)
```

SQL:

LinkedIn@AshishZope

```
SELECT *  
FROM employees VERSION AS OF 5;
```

Useful for:

- Auditing
- Recovery
- Debugging
- Historical analysis

50. MERGE

Extremely important in Data Engineering.

Example:

```
MERGE INTO target t
USING source s
ON t.employee_id = s.employee_id

WHEN MATCHED THEN
  UPDATE SET *

WHEN NOT MATCHED THEN
  INSERT *;
```

Common use:

```
Source
↓
New + Updated Records
↓
MERGE
↓
Target Delta Table
```

51. SCD

Slowly Changing Dimensions.

SCD Type 1

Overwrite old value.

LinkedIn@AshishZope

Old:
Salary = 50K

New:
Salary = 60K

Target:
Salary = 60K

No history.

SCD Type 2

Maintain history.

Typical columns:

LinkedIn@AshishZope

employee_id
salary
start_date
end_date
is_current

Example:

LinkedIn@AshishZope

101 | 50K | Jan | Mar | false
101 | 60K | Apr | NULL | true

52. Schema Evolution

Allows compatible changes to schema.

Example:

```
Old:
id
name

New:
id
name
department
```

LinkedIn@AshishZope

Delta can support schema evolution depending on configuration.

Interview theory

Schema evolution is important in production because raw data sources often change over time. A robust pipeline must handle new columns, nullable values, and backward compatibility without breaking downstream jobs.

53. Partitioning

Partition large datasets by frequently filtered columns.

Example:

```
df.write \
  .partitionBy("year", "month") \
  .format("delta") \
  .save(path)
```

LinkedIn@AshishZope

Directory:

```
year=2026/
month=10/
```

LinkedIn@AshishZope

Benefits:

- Partition pruning
- Less data read

Don't partition blindly on high-cardinality columns.

Bad candidates can include:

```
customer_id  
transaction_id
```

LinkedIn@AshishZope

when they have extremely high cardinality.

54. Bucketing

Bucketing distributes data into fixed buckets based on a column.

Useful for certain join/query patterns.

Concept:

```
employee_id  
  ↓  
Hash  
  ↓  
Bucket 1  
Bucket 2  
Bucket 3  
...
```

LinkedIn@AshishZope

Partitioning and bucketing are different concepts.

55. Data Skew

Data skew occurs when some partitions contain much more data than others.

Example:

```
Partition 1 → 10 MB  
Partition 2 → 12 MB  
Partition 3 → 11 MB  
Partition 4 → 900 MB
```

One task becomes a straggler.

Common causes:

- Highly skewed join key
- Popular customer
- NULL key
- Uneven data distribution

Solutions:

- Broadcast join
- Salting
- AQE
- Repartitioning
- Better join strategy

56. Salting

Used to handle skewed keys.

Example:

```
customer_id = 100
```

has millions of records.

Add random salt:

```
100_1  
100_2  
100_3  
...
```

Then distribute work across partitions.

57. Adaptive Query Execution (AQE)

AQE allows Spark to optimize execution based on runtime statistics.

Can help with:

- Dynamic partition coalescing
- Skew join handling
- Join strategy optimization

Important interview topic.

58. Predicate Pushdown

Instead of reading everything:

```
Read entire dataset  
  ↓  
Filter
```

Spark/storage can often push filter closer to the data source:

```
Filter  
  ↓  
Read only required data
```

Example:

```
df.filter(col("salary") > 1000000)
```

This can reduce I/O.

59. Column Pruning

If you need:

```
df.select("employee_id", "salary")
```

Spark can avoid reading unnecessary columns from suitable columnar sources.

60. Join Strategies

Common Spark strategies:

Broadcast Hash Join

Small table + large table.

Sort Merge Join

Common for large datasets.

Shuffle Hash Join

Can be used in suitable cases.

Broadcast Nested Loop Join

Special cases; potentially expensive.

Interview question:

How do you optimize a large-large join?

Discuss:

- Filter early
- Select required columns
- Check skew
- Repartition appropriately
- AQE
- Join strategy
- Avoid unnecessary shuffle

61. Spark SQL

Register temporary view:

```
df.createOrReplaceTempView("employees")
```

LinkedIn@AshishZope

Query:

```
SELECT department,  
       AVG(salary)  
FROM employees  
GROUP BY department;
```

LinkedIn@AshishZope

Run:

```
spark.sql("""  
SELECT *  
FROM employees  
WHERE salary > 1000000  
""").show()
```

LinkedIn@AshishZope

62. Temporary View vs Global Temporary View

Temporary:

```
createOrReplaceTempView()
```

LinkedIn@AshishZope

Available within current Spark session.

Global temporary view:

```
createOrReplaceGlobalTempView()
```

LinkedIn@AshishZope

Access:

```
SELECT *  
FROM global_temp.employees;
```

LinkedIn@AshishZope

63. Databricks Architecture

Typical Azure Databricks pipeline:

```
Source
↓
ADLS Gen2
↓
Databricks
↓
PySpark
↓
Bronze
↓
Silver
↓
Gold
↓
Power BI / Consumers
```

64. Medallion Architecture

Bronze

Raw data.

Source → Bronze

Minimal transformation.

Silver

Cleaned/conformed data.

Bronze → Silver

Typical operations:

- Remove duplicates
- Handle nulls

- Standardize formats
- Data validation
- Join/reference data

Gold

Business-ready data.

Silver → Gold

LinkedIn@AshishZope

Examples:

- KPIs
- Aggregates
- Reporting tables
- Business metrics

65. Production PySpark Pipeline

A good pipeline typically follows:

```
Read
↓
Validate
↓
Filter
↓
Transform
↓
Join
↓
Aggregate
↓
Optimize
↓
Write
↓
Monitor
```

DATA ENGINEER

LinkedIn@AshishZope

66. Incremental Processing

Instead of processing everything every day:

Full load:
1 TB → process 1 TB

LinkedIn@AshishZope

Incremental:

1 TB existing
+
10 GB new
↓
Process only 10 GB

LinkedIn@AshishZope

Common techniques:

- Watermark
- Last modified timestamp
- CDC
- Delta Change Data Feed
- Source-specific incremental keys

67. Checkpoint vs Cache

Cache

Used to improve performance by reusing computed data.

Checkpoint

Used to truncate lineage and provide recovery/state semantics depending on use case.

Streaming applications commonly use checkpoints for maintaining progress/state.

68. Structured Streaming

Spark Structured Streaming processes continuously arriving data using DataFrame/Dataset APIs.

Example:

```
stream_df = spark.readStream \
    .format("cloudFiles") \
    .option("cloudFiles.format", "json") \
    .load(input_path)
```

LinkedIn@AshishZope

Write:

```
stream_df.writeStream \
    .format("delta") \
    .option("checkpointLocation", checkpoint_path) \
    .start(output_path)
```

LinkedIn@AshishZope

69. Output Modes

Structured Streaming supports:

```
append
complete
update
```

LinkedIn@AshishZope

Append

Only new rows.

Complete

Entire result table.

Update

Only changed rows.

70. Watermark

Used to handle late-arriving data and limit state retention.

Concept:

```
Event time
  ↓
Watermark
  ↓
Older events eventually excluded from state
```

LinkedIn@AshishZope

Example:

```
df.withWatermark(
    "event_time",
    "10 minutes"
)
```

LinkedIn@AshishZope

71. Exactly-Once Concept

In distributed systems, exactly-once processing depends on the entire architecture and sink semantics.

Delta Lake + Structured Streaming can provide strong exactly-once processing semantics in appropriate configurations.

Interview question:

Exactly once vs at least once?

At least once:
A record may be processed multiple times.

At most once:
A record may be lost.

Exactly once:
Each record's effect is committed once.

72. Spark Performance Optimization

Remember this checklist:

1. Filter early
2. Select only required columns
3. Avoid unnecessary shuffles
4. Broadcast small tables
5. Handle data skew
6. Choose partitions carefully
7. Use Parquet/Delta
8. Avoid unnecessary UDFs
9. Cache only reused data
10. Use AQE
11. Avoid collect()
12. Check explain()

73. Small Files Problem

Example:

1 million files × 10 KB

can be inefficient compared with fewer appropriately sized files.

Causes:

- Too many partitions

- Frequent small writes
- Streaming micro-batches
- Poor partition design

Solutions:

- Optimize file layout
- Compact files
- Avoid excessive partition counts
- Use appropriate write strategies

74. Common Databricks Concepts

Know these:

Workspace
Notebook
Cluster
Compute
Serverless
Dedicated Compute
Catalog
Schema
Table
View
Volume
Jobs
Workflows
Unity Catalog
Delta Lake
DBFS / cloud storage concepts

LinkedIn@AshishZope

75. Unity Catalog

Three-level namespace:

catalog.schema.table

LinkedIn@AshishZope

Example:

```
SELECT *  
FROM employee_catalog.employee_db.employees;
```

LinkedIn@AshishZope

You have been using this structure in Databricks.

76. Managed vs External Tables

Managed table

Databricks manages table storage/location lifecycle according to the platform/table configuration.

External table

Data exists at an externally managed storage location.

Important interview topic:

Metadata and data lifecycle can differ between managed and external tables.

77. Views

View

Logical query.

```
CREATE VIEW employee_view AS  
SELECT *  
FROM employees;
```

LinkedIn@AshishZope

Materialized view

Stores/maintains materialized results depending on platform capabilities and configuration.

78. Common PySpark Functions You Must Know



```
col
lit
when
otherwise

sum
avg
min
max
count
countDistinct

row_number
rank
dense_rank
lag
lead

concat
concat_ws
split
trim
upper
lower
regexp_replace

to_date
to_timestamp
date_format
datediff
date_add
date_sub

coalesce
nullif

explode
explode_outer
posexplode

array
array_contains
collect_list
collect_set

broadcast
```

79. explode()

Array:

```
["SQL", "PySpark", "Python"]
```

LinkedIn@AshishZope

After:

```
explode("skills")
```

LinkedIn@AshishZope

becomes:

```
SQL
PySpark
Python
```

LinkedIn@AshishZope

Example:

```
df.select(
    "employee_id",
    explode("skills").alias("skill")
)
```

LinkedIn@AshishZope

80. collect_list vs collect_set

```
collect_list("skill")
```

LinkedIn@AshishZope

keeps duplicates.

```
collect_set("skill")
```

LinkedIn@AshishZope

removes duplicates.

81. JSON / Nested Data

Suppose:

```
{  
  "employee_id": 101,  
  "address": {  
    "city": "Pune",  
    "state": "Maharashtra"  
  }  
}
```

LinkedIn@AshishZope

Access:

```
df.select(  
  "employee_id",  
  "address.city",  
  "address.state"  
)
```

LinkedIn@AshishZope

82. Handling Arrays and Structs

Struct:

```
df.select("address.city")
```

LinkedIn@AshishZope

Array:

```
df.select(  
  explode("skills").alias("skill")  
)
```

LinkedIn@AshishZope

This is commonly tested in interviews.

83. Error Handling in Production

Use:

```
try:
    ...
except Exception as e:
    ...
```

LinkedIn@AshishZope

But don't silently swallow errors.

Good pipeline:

```
Input validation
↓
Transformation
↓
Data quality checks
↓
Write
↓
Logging
↓
Alerting
```

LinkedIn@AshishZope

84. Data Quality Checks

Examples:

```
Null checks
Duplicate checks
Schema validation
Row count validation
Referential integrity
Business-rule validation
```

LinkedIn@AshishZope

Example:

```
if df.filter(col("employee_id").isNull()).count() > 0:  
    raise Exception("Invalid employee IDs")
```

LinkedIn@AshishZope

85. Idempotency

Very important production concept.

An idempotent pipeline can be rerun without producing incorrect duplicate results.

Bad:

```
Run 1 → 100 rows  
Run 2 → same 100 rows appended again  
→ 200 rows
```

LinkedIn@AshishZope

Better:

```
MERGE / overwrite partition / deterministic processing
```

LinkedIn@AshishZope

so rerunning gives the same correct result.

86. PySpark Coding Pattern

A clean production structure:

```
from pyspark.sql import functions as F

def transform(df):

    return (
        df
        .filter(F.col("salary").isNotNull())
        .withColumn(
            "salary_category",
            F.when(F.col("salary") > 1000000, "HIGH")
              .otherwise("NORMAL")
        )
        .select(
            "employee_id",
            "salary",
            "salary_category"
        )
    )
```

Then:

```
df_transformed = transform(df)
```

87. Most Important Interview Questions

Basic

1. What is PySpark?
2. What is Spark?
3. What is SparkSession?
4. What is DataFrame?
5. DataFrame vs RDD?
6. What is lazy evaluation?
7. Transformation vs action?
8. What is a partition?
9. What is shuffle?
10. Narrow vs wide transformation?

88. Intermediate Interview Questions

11. repartition vs coalesce?
12. cache vs persist?
13. What is broadcast join?
14. What is data skew?
15. How do you handle skew?
16. What is Catalyst?
17. What is Tungsten?
18. What is predicate pushdown?
19. What is column pruning?
20. How does Spark optimize queries?

89. Advanced Interview Questions

21. Explain Spark architecture.
22. Driver vs executor?
23. What happens internally when you call `show()`?
24. Explain job → stage → task.
25. What causes a new stage?
26. What is shuffle?
27. How does Spark handle failures?
28. What is speculative execution?
29. Explain AQE.
30. Explain join strategies.
31. How would you optimize a slow join?
32. How would you handle 1 TB dataset?
33. How would you debug a slow Spark job?
34. How do you identify data skew?
35. How do you optimize a PySpark pipeline?

90. Scenario-Based Interview Questions

Scenario 1

You have:

Employee = 500 million rows
Department = 100 rows

LinkedIn@AshishZope

How would you join?

Expected discussion:

Broadcast Department

LinkedIn@AshishZope

Scenario 2

One customer contains 50% of all records.

Problem?

Data skew

LinkedIn@AshishZope

Possible solution:

Salting
AQE
Broadcast if applicable
Better partitioning

LinkedIn@AshishZope

Scenario 3

Your Spark job is very slow.

Check:

- Spark UI
- Explain plan
- Shuffle
- Skew
- Partitions
- Join strategy
- Input size
- File count
- Caching
- UDFs

Scenario 4

You have 10,000 tiny output files.

Problem:

Small files problem

Solutions:

- Reduce partitions
- Compact files
- Optimize write strategy

Scenario 5

Pipeline failed after processing 70%.

How can you safely rerun?

Discuss:

Checkpointing where applicable
 Idempotency
 Incremental processing
 MERGE
 Transactional Delta writes

91. SQL → PySpark Mapping

SQL	PySpark
SELECT	select
WHERE	filter / where
GROUP BY	groupBy
ORDER BY	orderBy
JOIN	join
CASE	when
DISTINCT	distinct
COUNT	count
AVG	avg
SUM	sum
ROW_NUMBER	row_number
RANK	rank
LAG	lag
LEAD	lead

92. Your Core PySpark Mental Model

Whenever you see a PySpark problem, think:

1. What is the input?
↓
2. What is the schema?
↓
3. What transformation is required?
↓
4. Will it cause shuffle?
↓
5. How many partitions?
↓
6. Is there data skew?
↓
7. Can I filter/project early?
↓
8. Can I broadcast?
↓
9. Is caching useful?
↓
10. What does explain() show?
↓
11. How will I write the result?
↓
12. Is the pipeline idempotent?

93. 30-Second Interview Revision

If an interviewer asks:

"Explain PySpark."

Say:

PySpark is the Python API for Apache Spark, a distributed processing framework used for large-scale data processing. It provides DataFrame and SQL APIs, supports distributed transformations and actions, and uses lazy evaluation and query optimization to execute workloads efficiently across a cluster.

"How do you optimize PySpark?"

Think:

```
Filter early
↓
Select required columns
↓
Avoid unnecessary shuffle
↓
Broadcast small datasets
↓
Handle skew
↓
Tune partitions
↓
Use Parquet/Delta
↓
Avoid unnecessary UDFs
↓
Cache reused data
↓
Use AQE
↓
Inspect Spark UI + explain()
```

94. Final PySpark Revision Roadmap

You should be able to explain this entire chain:

Python
↓
PySpark
↓
SparkSession
↓
DataFrame
↓
Schema
↓
select / filter / withColumn
↓
Functions
↓
Aggregations
↓
Joins
↓
Windows
↓
Transformations
↓
Actions
↓
Lazy Evaluation
↓
Partitions
↓
Shuffle
↓
Stages
↓
Tasks
↓
Catalyst
↓
Tungsten
↓
AQE
↓
Optimization
↓
Parquet
↓
Delta Lake
↓
MERGE
↓
SCD
↓
Incremental Processing
↓

Structured Streaming
↓
Databricks
↓
Production Pipelines

95. High-Probability Interview Theory

If you want a compact interview-focused theory recap, remember these points:

1. Why PySpark?

Because Python alone cannot efficiently process huge distributed datasets, while Spark provides parallel, distributed processing with fault tolerance and optimization.

2. Why DataFrame over RDD?

Because DataFrame gives schema, optimization, SQL compatibility, and better performance.

3. Why lazy evaluation matters?

Because it allows Spark to plan optimizations before execution, reducing workload and improving performance.

4. Why shuffle is expensive?

Because it moves data across the cluster and can create bottlenecks, skew, and extra stages.

5. Why joins need optimization?

Because joins can trigger large shuffle operations; broadcast joins, filtering, and partitioning reduce cost.

6. Why data quality and idempotency matter?

Because production pipelines must be repeatable, reliable, and safe to rerun.

7. Why Delta is important?

Because it adds ACID guarantees, versioning, time travel, and merge support, which are essential for production data engineering.

8. Why Apache Spark matters in interviews?

Because interviewers are not only testing syntax; they want to see whether you understood distributed systems concepts, scalability, optimization, and production debugging.

96. Must-Remember Interview Checklist

Before your interview, make sure you can explain **without looking at notes**:

- ☐ Spark architecture
- ☐ Driver / Executor
- ☐ SparkSession
- ☐ DataFrame
- ☐ Schema
- ☐ Transformations
- ☐ Actions
- ☐ Lazy evaluation
- ☐ Narrow / Wide transformations
- ☐ Partitions
- ☐ Shuffle
- ☐ repartition / coalesce
- ☐ groupBy
- ☐ joins
- ☐ broadcast join
- ☐ window functions
- ☐ UDF / Pandas UDF
- ☐ RDD vs DataFrame
- ☐ Catalyst
- ☐ Tungsten
- ☐ AQE
- ☐ Predicate pushdown
- ☐ Column pruning
- ☐ Data skew

- ☐ Salting
- ☐ Cache / persist
- ☐ Explain plan
- ☐ Parquet
- ☐ Delta Lake
- ☐ Time Travel
- ☐ MERGE
- ☐ SCD Type 1 / Type 2
- ☐ Partitioning
- ☐ Small files
- ☐ Incremental processing
- ☐ Idempotency
- ☐ Structured Streaming
- ☐ Watermark
- ☐ Checkpoint
- ☐ Medallion architecture
- ☐ Unity Catalog
- ☐ Databricks production architecture
- ☐ Performance optimization
- ☐ Debugging Spark jobs
- ☐ Scenario-based questions

The 5 concepts you should master most deeply

If you have very limited revision time, focus especially on:

1. Transformations + Lazy Evaluation

2. Partitions + Shuffle + Stages + Tasks

3. Joins + Broadcast + Data Skew

4. Window Functions + Aggregations

5. Delta Lake + MERGE + Incremental Processing + SCD

These five areas connect a large part of real-world PySpark Data Engineering work and interview questions.

97. Final 60-Second Interview Answer Template

Use this template when asked broad PySpark questions:

PySpark is the Python API for Apache Spark, designed for distributed processing of large datasets. It uses DataFrames and Spark SQL to express transformations and actions, while Spark optimizes the execution plan and distributes work across executors. Key concepts include lazy evaluation, partitions, shuffles, joins, caching, and window functions. In production, we also focus on data quality, idempotency, Delta Lake, and performance tuning. The goal is to process data efficiently, reliably, and at scale.

98. Common Follow-Up Interview Questions and Strong Answers

Q: What is the biggest difference between Spark and Python?

A: Python processes data serially in one process, while Spark distributes data across a cluster and executes work in parallel.

Q: Why is Spark faster than Python loops?

A: Because Spark distributes work across executors and uses optimized execution plans, while Python loops run in a single process.

Q: Why is lazy evaluation important?

A: It allows optimization before execution and avoids unnecessary work.

Q: What causes a slow Spark job?

A: Often shuffle, skew, large file counts, bad partitioning, expensive UDFs, or poor joins.

Q: How do you debug a bad Spark performance issue?

A: Use `explain()`, inspect the Spark UI, identify stages with high shuffle or skew, filter early, select only needed columns, and check partition counts.

Q: Why is Delta Lake used?

A: It provides reliable transactional storage, schema handling, time travel, and merge capability for production data engineering.

99. One-Line Revision Summary

If you remember only a few things, remember:

Spark = parallel distributed processing
DataFrame = structured abstraction for data work
Lazy evaluation = optimize before execution
Shuffle = expensive cross-partition movement
Broadcast join = optimize small lookup joins
Delta = production-grade reliable table format
Interview success = explain concepts clearly, not just code syntax

LinkedIn@AshishZope